

The `posim` Command Language

Complete Grammar & Notebook Guide

physical_object_simulator

July 21, 2026

Contents

1	What is this document about?	2
2	Lexical structure (what the lexer sees)	2
2.1	Token kinds	2
2.2	Keywords	3
2.3	Magics	3
3	The grammar (what the parser accepts)	3
4	The type system (what values exist)	5
5	Command semantics	5
5.1	NEW — create an object	5
5.2	SET / GET — write and read any field	6
5.3	STEP and RUN — integrate (always via SUNDIALS)	8
5.4	METHOD — choose the integrator	8
5.5	Observables, bookkeeping, help	9
5.6	COLLIDE and CONTACTS — rigid-body collisions	9
5.7	BOX — the rigid bounding box	10
6	The notebook (cells, editing, magics)	11
6.1	Cells	11
6.2	State is cumulative	11
6.3	Magics	11
6.4	Scripts	11
7	JupyterLab in one paragraph	11
8	The stack machine (what runs your line)	12
9	Thirteen worked examples	12
9.1	Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass	12
9.2	Example 2 — a thrown ball vs. the textbook formula	13
9.3	Example 3 — cyclotron motion: expressions as arguments	13
9.4	Example 4 — the tennis-racket theorem (Dzhanibekov effect)	14

9.5	Example 5 — three bodies, then surgery with DEL	14
9.6	Example 6 — magnetic torque (and why ENERGY grows)	15
9.7	Example 7 — fixing a typo in an old cell with %edit	15
9.8	Example 8 — checking the simulator against itself	16
9.9	Example 9 — hand-built tensors and a quaternion orientation	16
9.10	Example 10 — reproducibility: %save, %reset, %load	17
9.11	Example 11 — watching a Kepler orbit live, then running it backward	17
9.12	Example 12 — driving the camera from the notebook	18
9.13	Example 13 — the box of shapes: every body type in a rigid, infinitely massive box	19

10 Error message tour 21

1 What is this document about?

posim is the front end of the `physical_object_simulator`. Instead of writing Rust code to create objects and run physics, you type short *commands* — one per line — into a *notebook*. Each command line is:

1. chopped into *tokens* by a *lexer* (like the classic Unix tool `lex/flex`);
2. checked against a *grammar* and compiled into a small program by a *parser* (like `yacc/bison`);
3. executed by a *stack machine* — a tiny computer whose only memory is a stack of values — which reads and writes the simulator exclusively through its public `get/set` functions.

You never see steps 1–3; you type a line, press Enter (or shift-enter in JupyterLab), and see the result. This document specifies exactly what you may type.

There are three ways to talk to the same language:

mode	start with	who it is for
notebook REPL	<code>cargo run</code> (or <code>posim</code>)	humans at a terminal
script	<code>posim -script file</code>	batch runs, reproducibility
machine	<code>posim -machine</code>	programs, and the JupyterLab kernel

2 Lexical structure (what the lexer sees)

A command line is a sequence of *tokens* separated by optional whitespace. Everything from a `#` to the end of the line is a *comment* and is ignored.

2.1 Token kinds

token	examples	rules
keyword	NEW, set, Run	case-insensitive; list in §2.2
identifier	obj0, position	a letter or <code>_</code> , then letters, digits, <code>_</code>
number	2, 0.5, .5, 1e-3	64-bit float; scientific notation allowed; a leading <code>-</code> is <i>not</i> part of the number — it is the negation operator
punctuation	[] { } () , . =	brackets build vectors, braces enclose initializers, <code>.</code> builds dotted paths
operators	+ - * /	ordinary arithmetic

If the lexer meets a character it does not know (say @), it stops with an error naming the *column* (character position, starting at 1):

```
Err[1]: lexical error at column 10: unexpected character '@'
```

2.2 Keywords

NEW SET GET DEL DELETE LIST STEP RUN STEPS METHOD ADAMS BDF SPRK ENERGY COM MOMENTUM
ANGMOM LAPLACE HELP POINT SPHERE CUBOID TORUS DISK CYLINDER BOX RESET

and, for the graphical scene window (documented in `scene_info.pdf`; Examples 11–12 show live sessions):

SCENE CREATE CLOSE DESTROY TRANSLATE ROTATE ZOOM IN OUT HIDE SHOW REFRESH REDRAW START
STOP PAUSE REVERSE SET_TIME_STEP SETTIMESTEP STATUS EVENTS ALL

and, for rigid-body collisions (§5.6):

COLLIDE CONTACTS ON OFF

(DESTROY is an alias for CLOSE; SETTIMESTEP for SET_TIME_STEP; the spellings CUBE and DISC are lexer aliases for CUBOID and DISK. `collisions` is deliberately **not** a keyword so the field path `system.collisions` keeps its own spelling.)

Keywords are reserved *at the start of paths*, but **field names may reuse keyword spellings** — `obj0.momentum` and `system.method` work even though MOMENTUM and METHOD are keywords, because after a dot (or inside `NEW { ... }`) the parser accepts either an identifier or a keyword as a field name. The same holds for the scene keywords: making SHOW or IN reserved does not stop you from ever using those spellings as field names after a ..

2.3 Magics

A line whose first character is % is a *notebook magic* (§6.3) and is handled by the notebook itself; it never reaches the lexer.

3 The grammar (what the parser accepts)

The grammar in EBNF (Extended Backus–Naur Form: `[x]` means optional, `{x}` means zero-or-more repetitions, `|` separates alternatives):

```
line      := command | magic ;

command   := "NEW" shape [ "{" init { "," init } "}" ]
           | "SET" path "=" expr
           | "GET" path
           | "DEL" NUMBER
           | "LIST"
           | "STEP" expr                                (* advance time by dt      *)
           | "RUN" expr [ "STEPS" NUMBER ]              (* advance by t, n outputs *)
           | "METHOD" ( "ADAMS" | "BDF" | "SPRK" IDENT [ NUMBER ] )
           | "ENERGY" | "COM" | "MOMENTUM" | "ANGMOM"
           | "LAPLACE" NUMBER
```

```

| "RESET"
| "HELP"
| "SCENE" scenecmd (* graphical scene window *)
| "COLLIDE" [ "ON" | "OFF" ] (* bare: report status *)
| "CONTACTS" (* contacts of last run *)
| "BOX" [ "OFF" | expr ] (* rigid bounding box:
                           expr = inner side
                           length; bare = status;
                           OFF removes it *)
| expr ; (* bare expression *)

scenecmd := "CREATE" [ NUMBER ] (* open window [TCP port] *)
| "CLOSE" (* alias: DESTROY *)
| "TRANSLATE" term term [ term ] (* camera dx dy [dz] *)
| "ROTATE" term term (* camera dyaw dpitch, deg *)
| "ZOOM" ( "IN" | "OUT" | term ) (* factor > 1 zooms in *)
| "HIDE" [ NUMBER | "ALL" ] (* default: ALL *)
| "SHOW" [ NUMBER | "ALL" ]
| "REFRESH" (* re-sync from notebook *)
| "REDRAW" (* re-send whole scene *)
| "START" | "STOP" | "PAUSE" | "REVERSE"
| "SET_TIME_STEP" term
| "STATUS" | "EVENTS" ;

shape := "POINT" | "SPHERE" | "CUBOID" | "TORUS" | "DISK" | "CYLINDER" ;
init := FIELD "=" expr ;

path := root "." FIELD [ "." COMPONENT ] ;
root := "objN" (* N = object index, e.g. obj0, obj12 *)
| "contactK" (* K = contact index of the last run *)
| "system" | "sys" ;
COMPONENT:= "x" | "y" | "z" | "w" ;

expr := term { ("+" | "-") term } ;
term := unary { ("*" | "/" ) unary } ;
unary := "-" unary | atom ;
atom := NUMBER
| "[" expr { "," expr } "]" (* vector/matrix literal *)
| "(" expr ")"
| FUNC "(" [ expr { "," expr } ] ")" (* builtin function call *)
| path
| "pi" | "tau" ;
FUNC := "dot" | "cross" | "norm" | "normalize"
| "sqrt" | "abs" | "sin" | "cos" | "exp" | "log" ;

```

Operator precedence is the usual one: `*` and `/` bind tighter than `+` and `-`; unary minus binds tightest; parentheses override everything.

One subtlety worth learning early: GET takes *only a path*. To do arithmetic with a field, use a *bare expression*:

```

get obj0.position.x - 5      # ERROR -- GET wants just a path
obj0.position.x - 5         # OK -- bare expression

```

A second subtlety — SCENE arguments are *terms*, not full expressions. The scene commands take several numbers in a row with no commas between them. If those numbers were parsed as full expressions, `scene rotate 15 -5` would be read as *one* argument $15 - 5 = 10$ (whitespace never matters to the lexer) and the command would then be missing its second argument. So scene arguments stop one level lower in the grammar, at **term**: `-5` is negative five, `2*2` and `1/3` still work, but a sum or difference must be parenthesized:

<code>scene rotate 15 -5</code>	<code># yaw +15 deg, pitch -5 deg (two arguments)</code>
<code>scene zoom (1 + 0.5)</code>	<code># a sum needs parentheses</code>
<code>scene translate 2*2 0 1/2</code>	<code># * and / are fine unparenthesized</code>

4 The type system (what values exist)

Every expression evaluates to one of these *values*:

type	written as	notes
number	2, -0.5, 1e-3	64-bit float
vec3	[1, 2, 3]	exactly 3 numeric entries
quaternion	[w, x, y, z]	exactly 4 numeric entries, w first ; assigned to orientation it is renormalized to unit length
mat3	[[a,b,c],[d,e,f],[g,h,i]]	3 rows of 3 numbers
string	(produced, not typed)	results like <code>obj0</code> or run summaries

Bracket literals are *shape-directed*: 3 numbers make a `vec3`, 4 numbers make a quaternion, 3 `vec3`s make a `mat3`.

Arithmetic is *type-checked*:

operation	allowed
<code>+</code> , <code>-</code>	<code>number±number</code> , <code>vec3±vec3</code> , <code>quat±quat</code>
<code>*</code>	<code>number·number</code> , <code>number·vec3</code> , <code>number·quat</code> , <code>number·mat3</code> , <code>mat3·vec3</code> , <code>mat3·mat3</code> , <code>quat·quat</code> (Hamil- ton product)
<code>/</code>	<code>number/number</code> , <code>vec3/number</code>
<code>vec3*vec3</code>	forbidden — the error tells you to use <code>dot()</code> or <code>cross()</code>

Builtins: `dot(a,b)`, `cross(a,b)`, `norm(v)` (length; also quaternions and numbers), `normalize(v)`, and scalar `sqrt` `abs` `sin` `cos` `exp` `log`. Constants: `pi`, `tau` ($= 2\pi$).

5 Command semantics

5.1 NEW — create an object

<code>NEW POINT</code>	<code>{ field = expr, ... }</code>	
<code>NEW SPHERE</code>	<code>{ field = expr, ... }</code>	
<code>NEW CUBOID</code>	<code>{ field = expr, ... }</code>	(alias: <code>NEW CUBE</code>)
<code>NEW TORUS</code>	<code>{ field = expr, ... }</code>	
<code>NEW DISK</code>	<code>{ field = expr, ... }</code>	(alias: <code>NEW DISC</code>)
<code>NEW CYLINDER</code>	<code>{ field = expr, ... }</code>	

Creates one *physical object* and prints its handle (obj0, obj1, ..., numbered by position). Defaults: mass 1, at the origin, at rest, no charge; sphere radius 1; cuboid half-extents [1,1,1]; torus ring_radius 1, tube_radius 0.25; disk radius 1; cylinder radius 0.5, half_height 1.

Initializer fields: mass, charge, position, velocity, momentum, orientation, angular_velocity, angular_momentum, radius (spheres, disks, cylinders), half_extents (cuboids), ring_radius, tube_radius (tori — or the inner_radius + outer_radius pair, see below), height, half_height (cylinders; HEIGHT is the full height = 2*half_height), inertia_tensor, inverse_inertia_tensor, magnetic_moment_tensor, force, torque, id, inverse_mass.

A torus can be sized two ways: directly (ring_radius c = the radius of the circle traced by the tube's center, tube_radius a = the tube's own radius) or by the **inner_radius + outer_radius pair** (the hole's radius and the outermost radius: inner = $c - a$, outer = $c + a$). Inside a NEW initializer list the torus geometry is **deferred**: the four radius fields are collected and resolved *once*, at the end of the list — ring_radius/tube_radius are applied first, then inner_radius/outer_radius override the derived pair — and the result is validated once, against the *final* values (a bad pair fails with torus needs $0 <= \text{inner} < \text{outer}$ (got inner = 2, outer = 1)). Giving both members of the pair is therefore **genuinely order-independent**: { inner_radius = 1, outer_radius = 2 } yields ring 1.5, tube 0.5 whichever comes first, and a pair that shrinks the default torus — { outer_radius = 0.5, inner_radius = 0.2 } — works in either order too (checking each write against the half-updated default would have refused one of the two orders). inner_radius = 0 is legal — that is the **horn torus**, whose tube touches the axis — on NEW and on SET alike.

Four guarantees make initializers forgiving:

1. **Order does not matter for velocities.** velocity and angular_velocity are applied *after* mass and inertia are final, so { velocity = [1,0,0], mass = 2 } still yields momentum [2,0,0].
2. **Inertia is computed for you.** For every extended shape the inertia tensor is recomputed from the final mass and shape (solid sphere $\frac{2}{5}mr^2$; cuboid $\frac{m}{3}(h_y^2 + h_z^2)$ etc.; torus $I_z = m(c^2 + \frac{3}{4}a^2)$, $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$; disk $I_z = \frac{1}{2}ma^2$, $I_{xy} = \frac{1}{4}ma^2$; cylinder $I_z = \frac{1}{2}mr^2$, $I_{xy} = m(3r^2 + 4h^2)/12$ with h the half-height) — *unless* you supplied inertia_tensor yourself, in which case yours is kept.
3. **Coupled fields stay consistent** (see SET below).
4. **NEW is transactional.** If any initializer — or the final geometry validation — fails, the half-built object is removed before the error is reported: a failing NEW never leaves a ghost behind, and system.count and the objN numbering are exactly what they were before the command.

5.2 SET / GET — write and read any field

A *path* is objN.field or system.field, optionally followed by a component .x .y .z (vectors) or .w .x .y .z (quaternions). Component writes are safe read-modify-write operations through the full field's get/set pair.

Object fields (R = readable, W = writable):

field	type	R/W	meaning
-------	------	-----	---------

id	number	RW	user label (no effect on physics)
mass	number	RW	writing also updates <code>inverse_mass</code> ($m \leq 0 \Rightarrow 1/m := 0$)
inverse_mass	number	RW	writing back-computes <code>mass</code> ; 0 makes the body static
charge	number	RW	Coulombs
position, pos	vec3	RW	world position
velocity, vel	vec3	RW	<i>derived</i> : reads p/m , writes $p := mv$
momentum	vec3	RW	canonical stored linear state
orientation	quat	RW	renormalized on write
angular_velocity	vec3	RW	<i>derived</i> : $\omega = R I^{-1} R^T L$; writes $L := R I R^T \omega$
angular_momentum	vec3	RW	canonical stored angular state (spin)
inertia_tensor	mat3	RW	body frame; write updates the inverse (singular $\rightarrow$ zero inverse = cannot rotate)
inverse_inertia_tensor	mat3	RW	write back-computes the tensor
magnetic_moment_tensor	mat3	RW	torque $\tau = (R M R^T) B$
radius	number	RW	spheres, disks, cylinders; write recomputes inertia and keeps the shape family (a disk stays a disk, a cylinder keeps its height); a torus is refused — obj6 is a torus — set <code>ring_radius/tube_radius</code> or <code>inner_radius/outer_radius</code> instead of <code>radius</code> — rather than silently becoming a sphere
half_extents	vec3	RW	cuboids only; write recomputes inertia
ring_radius, tube_radius	number	RW	tori only; write recomputes inertia
inner_radius, outer_radius	number	RW	tori only, <i>derived</i> ($\text{inner} = c - a$, $\text{outer} = c + a$); writing one holds the other fixed; inner_radius accepts ≥ 0 (0 = the horn torus; the other radii must be > 0)
height, half_height	number	RW	cylinders only; height is the full height (= $2 \cdot \text{half_height}$); write recomputes inertia
boundary, shape	string	R	description of the shape
kinetic_energy, energy	number	R	$\frac{1}{2} m v ^2 + \frac{1}{2} \omega \cdot L$
force	vec3	RW	constant external force during STEP/RUN
torque	vec3	RW	constant external torque
restitution	number	RW	collision bounciness $e \in [0, 1]$ (default 1 = elastic; a pair uses $\min(e_i, e_j)$)

System fields:

field	type	R/W	meaning
<code>g_constant, g</code>	number	RW	Newton's G (default 1)
<code>softening</code>	number	RW	Plummer length ε (default 10^{-6}); forces use $(r^2 + \varepsilon^2)^{3/2}$
<code>uniform_gravity, gravity</code>	vec3	RW	constant acceleration field
<code>e_field</code>	vec3	RW	uniform E ; force qE
<code>b_field</code>	vec3	RW	uniform B ; force $qv \times B$, torque $(RMR^T)B$
<code>rtol, atol</code>	number	RW	CVODE tolerances (10^{-10} , 10^{-12})
<code>time, t</code>	number	RW	current simulation time
<code>method</code>	string	R	current integrator
<code>count, n</code>	number	R	number of objects
<code>collide</code>	number	R	1 when collision detection is on (switch with COLLIDE ON/OFF)
<code>contacts</code>	number	R	contacts recorded by the last STEP/RUN
<code>collisions</code>	number	R	running total of resolved impulses
<code>restitution_threshold</code>	number	RW	approach speeds below this bounce with $e = 0$ (default 10^{-3})
<code>contact_slop</code>	number	RW	tolerated overlap before positional projection (default 10^{-9})
<code>box</code>	number	R	inner side length of the rigid bounding box (§5.7); 0 = none. Create/remove with BOX

5.3 STEP and RUN — integrate (always via SUNDIALS)

STEP `dt` advances time by `dt`. RUN `t STEPS n` advances by `t`, stopping at `n` evenly spaced output points (default 10). Both accept full expressions (`run 2 * pi steps 8` is legal). The reply summarizes the run; `|dE/E|` is the relative change in total energy — your built-in sanity check. **All integration is performed by the pure-Rust SUNDIALS solvers**; there is no hand-rolled stepper.

5.4 METHOD — choose the integrator

METHOD ADAMS	(default; CVODE Adams-Moulton, adaptive)
METHOD BDF	(CVODE BDF -- stiff problems, fast gyration)
METHOD SPRK <table> [dt]	(ARKODE symplectic, fixed step; default dt 0.01)

Table names may be abbreviated: `leapfrog_2_2` becomes `ARKODE_SPRK_LEAPFROG_2_2`. Useful tables: `EULER_1_1`, `LEAPFROG_2_2` ($\equiv$ classic velocity Verlet), `MCLACHLAN_2_2/3_3/4_4/5_6`, `RUTH_3_3`, `YOSHIDA_6_8`.

SPRK requires a *separable* system: point-like translation only. If anything velocity-dependent or rotational is active (a `b_field`, a magnetic tensor, an external torque, a spinning rigid body), RUN refuses with an error *naming the offending feature*.

5.5 Observables, bookkeeping, help

command	prints
ENERGY	kinetic + softened pairwise potential + uniform-field potentials
COM	system center of mass
MOMENTUM	total linear momentum
ANGMOM	total angular momentum about the origin (orbital + spin)
LAPLACE n	Laplace–Runge–Lenz vector of object n about the center of mass, $k = G M_{\text{total}}$
LIST	one line per object
DEL n	removes object n (later objects renumber!)
RESET	wipes everything
HELP	the quick-reference card

5.6 COLLIDE and CONTACTS — rigid-body collisions

COLLIDE	(report: on/off, collidable pairs, impulses so far)
COLLIDE ON	(default - collisions are detected in EVERY scene)
COLLIDE OFF	(pure point-mass/gravity studies)
CONTACTS	(list every contact of the last STEP/RUN)

When collisions are on (the default) and the system contains at least one collidable pair (spheres, cuboids, or a point against either — two points cannot collide), **STEP** and **RUN** detect impacts **during** the time step by SUNDIALS *event rootfinding*: the integrator itself lands on the instant where a pair’s signed separation crosses zero, interpolated to solver precision. At that instant an impulse acts along the **contact normal** — the line of the action–reaction force pair — and integration continues. Nothing tunnels, and the reply reports what happened.

Every contact of the last **STEP**/**RUN** is recorded and exposed to the rest of the simulation through read-only `contactK` paths:

field	type	meaning
<code>contactK.i</code> , <code>contactK.j</code>	number	the colliding pair (indices, $i < j$)
<code>contactK.t</code>	number	the event time (the root the solver landed on)
<code>contactK.point</code>	vec3	world-space contact point
<code>contactK.normal</code>	vec3	unit normal, from obji toward objj — the action–reaction line; <code>objj</code> receives $+J\hat{n}$, <code>obji</code> receives $-J\hat{n}$
<code>contactK.depth</code>	number	penetration depth at the event (≈ 0)
<code>contactK.rel_vel_n</code>	number	approach speed along the normal (negative = approaching)
<code>contactK.impulse</code>	number	scalar impulse magnitude J

Response physics (full derivation in `collision_detection.pdf`): per-object **restitution** $e \in [0, 1]$ (1 = elastic default, 0 = perfectly plastic), pair-combined as $\min(e_i, e_j)$; impulse $J = -(1 + e)(\mathbf{v}_{\text{rel}} \cdot \hat{n}) / (\hat{n}^\top K \hat{n})$ with the effective-mass matrix K including the angular terms, applied through the canonical momentum/angular-momentum setters; static bodies (`inverse_mass` = 0) are immovable walls; approach speeds below `system.restitution_threshold` settle without bounce; leftover overlap beyond `system.contact_slop` is projected out. **SCENE** windows draw each contact normal as an arrow at the contact point (**Contacts** toolbar button or the **C** key).

5.7 BOX — the rigid bounding box

BOX <size>	(create: six static walls enclosing an inner size x size x size cube centered on the origin)
BOX OFF	(remove the box - its six walls are deleted)
BOX	(bare: report status)

BOX <size> builds a closed rigid room out of **six ordinary cuboid objects** — wall slabs behind the planes $x = \pm \text{size}/2$, $y = \pm \text{size}/2$, $z = \pm \text{size}/2$ (half-thickness $\text{size}/4$, centers at $\pm 3 \cdot \text{size}/4$, cross-sections wide enough to cover the corners) — and prints their handles:

In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4, obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off the inside faces

Infinite mass is exact, not approximate. The equations of motion never use the mass itself — only the *inverse* mass: velocity is $v = pm^{-1}$, and the collision impulse divides by $\hat{n}^\top K \hat{n} = m_i^{-1} + m_j^{-1} + (\text{angular terms})$ (§5.6). Each wall is created with `inverse_mass = 0` and a zero inverse inertia tensor, so it contributes exactly 0 to every impulse denominator and receives no state writes: bodies bounce off the inside faces elastically while the walls stay **bit-identically at rest**. One measurable consequence: system **momentum is not conserved** inside a box — the infinitely massive walls absorb it without moving (Example 13). LIST tags every wall [`wall: static, inverse_mass=0`] (and shows `mass=0` — the canonical stored quantity for a static body is the inverse).

Bookkeeping:

- GET `system.box` reads the inner side length; 0 means no box. The path is read-only — the box is created and removed by the command.
- A second BOX <size> **replaces** the existing box: the old walls are removed first and the reply notes (replacing the previous box).
- The walls are ordinary objects with ordinary indices: DEL on a wall deletes it like any other object **and dissolves the box** — `system.box` drops to 0 (five walls no longer enclose anything) — but the surviving slabs **stay tracked**: LIST keeps their [`wall: static, inverse_mass=0`] tag, and bare BOX reports `box: dissolved` (a wall was deleted; 5 tracked slab(s) remain - BOX <size> replaces them, BOX OFF removes them). The next BOX <size> removes the survivors *before* building the new box (the reply notes (replacing the previous box)) and BOX OFF simply deletes them — either way no orphan slab leaks. Wall indices are tracked through every DEL renumbering, so deleting a non-wall object never confuses the box.
- BOX OFF replies `box removed (6 wall(s) deleted, indices renumbered)` — or `box: none` if there was nothing to remove; bare BOX reports the size and wall handles, or `box: none` (BOX <size> creates one).
- A SCENE window draws the box as a **dashed interior wireframe**; the wall slabs themselves are not drawn as bodies. Creating a box while a window is open appends a reminder to the reply: (`scene window open: SCENE REFRESH` shows the box). RESET re-syncs an open

window to the now-empty system **and clears its box wireframe and wall flags** — no stale overlay survives the wipe.

6 The notebook (cells, editing, magics)

6.1 Cells

Start `posim` with no arguments. You get numbered prompts, exactly like Jupyter or Mathematica:

```
In[1]:= new sphere { mass = 2, radius = 0.5 }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[0.2, 0, 0], [0, 0.2, 0], [0, 0, 0.2]]
```

Pressing **Enter** **executes the cell** — in a plain terminal, Enter *is* shift-enter. Commands with no interesting result (like `SET`) print no `Out[n]`. Errors print `Err[n]:` and the session continues.

6.2 State is cumulative

Like Jupyter, the notebook has one live simulator state that every cell mutates in order. Re-running an old `NEW` cell creates a *new* object; it does not replace the old one. To rebuild cleanly, `%reset` and replay.

6.3 Magics

magic	effect
<code>%history</code>	lists every cell with input and output; failed cells marked !
<code>%edit n <text></code>	replaces cell <i>n</i> 's input and executes it as the next cell
<code>%rerun n</code>	executes cell <i>n</i> 's input again as a new cell
<code>%save <file></code>	writes all successful non-magic inputs to a replayable script
<code>%load <file></code>	replays a script file cell by cell
<code>%reset</code>	clears the simulator (history kept)
<code>%quit, %exit</code>	leave

A plain terminal has no cursor-addressable cells (`posim` uses only the Rust standard library — no raw terminal mode), so backward/forward movement is by these magics. **For true click-and-edit cells with shift-enter, use the JupyterLab front end (§7).**

6.4 Scripts

`posim -script file` executes a file of command lines (blank lines and `#` comments skipped), echoing `In[n]/Out[n]`, and exits nonzero if any cell failed — good for CI.

7 JupyterLab in one paragraph

The `jupyter/` directory ships a small *wrapper kernel*: JupyterLab starts a Python shim, the shim starts `posim -machine` (a JSON request/reply protocol over stdin/stdout), and each notebook cell you shift-enter is forwarded line-by-line as `{"op": "exec", "code": ...}`. Setup and tests:

jupyter/README.md. Everything in §§2–5 applies verbatim inside JupyterLab cells; multi-line cells run one line at a time, stopping at the first error.

8 The stack machine (what runs your line)

The parser emits *postfix instructions*; e.g. `set obj0.mass = 2 + 3 * 4` compiles to

```
Push 2
Push 3
Push 4
Mul          <- pops 3,4  pushes 12
Add          <- pops 2,12 pushes 14
Store obj0.mass  <- pops 14, calls set_mass(14)
```

The machine’s whole memory is a stack of typed values. Instructions: Push, Load, Store, Add Sub Mul Div Neg, PackList (literals), Call (builtins), NewObject/InitField/FinishNew, Delete, ListObjects, Step, Run, SetMethod, Energy, CenterOfMass, TotalMomentum, TotalAngularMomentum, Laplace, Reset, Help. Load/Store go *only* through the `physical_object` get/set API — the machine cannot corrupt simulator state, and every coupled invariant (`mass↔inverse`, `inertia↔inverse`, unit quaternions) is enforced on every write. When the program ends, the top of the stack becomes `Out[n]`.

9 Thirteen worked examples

All transcripts are genuine program output.

9.1 Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass

The Laplace–Runge–Lenz vector **A** points along a Kepler orbit’s major axis and is conserved *only* for a perfect $1/r^2$ force — the most delicate conservation test available. We build a “sun” so heavy the reduced problem is exact and integrate two orbits with a 4th-order *symplectic* method.

```
In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= laplace 1
Out[5]= [0.59999999974000001, 0, 0]
In[6]:= method sprk mclachlan_4_4 0.001
Out[6]= method = ARKODE SPRK ARKODE_SPRK_MCLACHLAN_4_4, fixed dt = 0.001
In[7]:= run 12.6 steps 2
Out[7]= t = 12.6 (12600 solver steps, 2 snapshots, |dE/E| = 9.237e-14)
In[8]:= laplace 1
Out[8]= [0.59999999974140128, -0.00000000034051644837163053, 0]
In[9]:= get obj1.position
Out[9]= [0.3964802853115723, 0.06706198328283366, 0]
```

What to notice. `g_constant = 1e-9` makes $GM_{\text{total}} = 1$ so the orbit has period 2π . LAPLACE 1 divided by `mk` is the *eccentricity vector*: $e = 0.6$ is read directly off `Out[5]`. After ~ 2 orbits the energy is conserved to 9×10^{-14} and **A** has rotated by only 3×10^{-10} — no spurious perihelion precession. Softening was zeroed because any $\varepsilon \neq 0$ slightly breaks the $1/r^2$ law and *would* precess the axis.

9.2 Example 2 — a thrown ball vs. the textbook formula

A projectile has the closed form $x(t) = v_x t$, $y(t) = y_0 + v_y t - \frac{1}{2}gt^2$. We fly a baseball for 6.12 s and subtract the formula *inside the notebook* using bare expressions.

```
In[1]:= new point { mass = 0.145, position = [0, 1, 0], velocity = [30, 30, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= run 6.12 steps 3
Out[3]= t = 6.12 (13 solver steps, 3 snapshots, |dE/E| = 4.740e-15)
In[4]:= get obj0.position.x
Out[4]= 183.59999999999999
In[5]:= 30 * 6.12
Out[5]= 183.6
In[6]:= obj0.position.x - 30 * 6.12
Out[6]= -0.000000000000008526512829121202
In[7]:= obj0.position.y - (1 + 30 * 6.12 - 9.81 / 2 * 6.12 * 6.12)
Out[7]= -0.00000000000004263256414560601
```

What to notice. Cells 6–7 are *bare expressions* mixing paths and arithmetic (`GET` itself would refuse the arithmetic). The trajectory matches the parabola to $\sim 10^{-13}$ — and the adaptive Adams integrator needed only **13 internal steps**, because for a polynomial solution its error estimator lets it take huge strides.

9.3 Example 3 — cyclotron motion: expressions as arguments

A charge in a uniform magnetic field circles with period $T = 2\pi m/(|q|B)$. We *compute* T *inside the RUN command*.

```
In[1]:= new sphere { mass = 2, radius = 0.1, charge = -1.5, velocity = [3, 0, 0] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0, 4]
In[3]:= method bdf
Out[3]= method = CVODE BDF
In[4]:= 2 * pi * 2 / (1.5 * 4)
Out[4]= 2.0943951023931953
In[5]:= run 2 * pi * 2 / (1.5 * 4) steps 8
Out[5]= t = 2.0943951023931953 (318 solver steps, 8 snapshots, |dE/E| = 1.444e-8)
In[6]:= get obj0.position
Out[6]= [0.00000000043106769672882073, 0.000000007220835657713453, 0]
In[7]:= norm(obj0.velocity)
Out[7]= 2.9999999783374913
```

What to notice. After one analytic period the particle is back at the origin to 7×10^{-9} , and `norm()` shows the speed unchanged — the magnetic force does no work. The Lorentz force $q \mathbf{v} \times \mathbf{B}$

is velocity-dependent, so the symplectic path is *not allowed* here; BDF is the right tool for fast gyration.

9.4 Example 4 — the tennis-racket theorem (Dzhanibekov effect)

A rigid body spun about its *intermediate* principal axis is unstable: a tiny wobble grows into a dramatic flip, yet energy and angular momentum stay conserved. Half-extends [0.5, 1, 2] give moments [5, 4.25, 1.25]; spinning about *y* (the middle value) triggers it.

```
In[1]:= new cuboid { mass = 3, half_extents = [0.5, 1, 2], angular_velocity = [0.01, 3,
0.01] }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[5, 0, 0], [0, 4.25, 0], [0, 0, 1.25]]
In[3]:= get obj0.angular_velocity
Out[3]= [0.010000000000000002, 3, 0.010000000000000002]
In[4]:= run 40 steps 4
Out[4]= t = 40 (2361 solver steps, 4 snapshots, |dE/E| = 5.605e-9)
In[5]:= get obj0.angular_velocity
Out[5]= [1.5428438559953521, 2.9947703802651175, -0.7871804456905063]
In[6]:= run 40 steps 4
Out[6]= t = 80 (2334 solver steps, 4 snapshots, |dE/E| = 5.332e-9)
In[7]:= get obj0.angular_velocity
Out[7]= [0.020666590902422656, 2.9999548562146483, 0.013346831306280834]
In[8]:= get obj0.angular_momentum
Out[8]= [0.05, 12.75, 0.0125]
```

What to notice. At $t = 40$ the wobble has exploded ($\omega_x \approx 1.54$, mid-flip); by $t = 80$ the body has completed the flip and returned to a clean spin. Throughout, the world-frame angular momentum (Out[8]) is exactly the initial $I\omega = [0.05, 12.75, 0.0125]$ — the solver integrates the full quaternion + angular-momentum state with energy error $\sim 5 \times 10^{-9}$.

9.5 Example 5 — three bodies, then surgery with DEL

```
In[1]:= new point { mass = 1, position = [1, 0, 0], velocity = [0, 1, 0] }
Out[1]= obj0
In[2]:= new point { mass = 4, position = [-1, 0, 0], velocity = [0, -0.25, 0] }
Out[2]= obj1
In[3]:= new point { mass = 256, position = [0, 8, 0], velocity = [0, 0, 0] }
Out[3]= obj2
In[4]:= set system.g_constant = 0.001
In[5]:= momentum
Out[5]= [0, 0, 0]
In[6]:= com
Out[6]= [-0.011494252873563218, 7.846743295019157, 0]
In[7]:= run 3 steps 3
Out[7]= t = 3 (84 solver steps, 3 snapshots, |dE/E| = 3.141e-10)
In[8]:= list
Out[8]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
obj2: point, mass=256, charge=0, pos=[-0.000017852126656859762, 7.999648688652149, 0]
In[9]:= del 2
```

```

Out[9]= deleted obj2; 2 object(s) remain (indices renumbered)
In[10]:= list
Out[10]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
In[11]:= momentum
Out[11]= [0.0021430470372574067, 0.061528401914275554, 0]

```

What to notice. Cells 1–3 chose velocities so the initial total momentum is exactly zero ($1 \cdot 1 + 4 \cdot (-0.25) = 0$), and it stays zero through the run. Deleting the heavy body removes its momentum: the remainder (Out[11]) is what the surviving pair carries — conservation applies to the system you keep. DEL renumbers later objects.

9.6 Example 6 — magnetic torque (and why ENERGY grows)

magnetic_moment_tensor M couples the world B field to torque $\tau = (RMR^T)B$. Unlike everything else, this coupling is *not derived from a potential*, so total energy is *expected* to change.

```

In[1]:= new sphere { mass = 1, radius = 0.5, magnetic_moment_tensor = [[0.2, 0, 0], [0,
    0.2, 0], [0, 0, 0.2]] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0.5, 0]
In[3]:= get obj0.angular_momentum
Out[3]= [0, 0, 0]
In[4]:= energy
Out[4]= 0
In[5]:= run 4 steps 4
Out[5]= t = 4 (214 solver steps, 4 snapshots, |dE/E| = 8.000e-1)
In[6]:= get obj0.angular_momentum
Out[6]= [0, 0.400000000000000085, 0]
In[7]:= get obj0.angular_velocity
Out[7]= [0, 4.000000000000009, 0]
In[8]:= energy
Out[8]= 0.80000000000000035

```

What to notice. Constant torque $\tau = MB = 0.2 \times 0.5 = 0.1$ about y gives $L(t) = 0.1t$: after 4 s, $L = 0.4$ exactly (the ODE is linear, so the solver nails it). Sphere inertia = $0.4 \cdot 1 \cdot 0.25 = 0.1$, so $\omega = L/I = 4$ and the rotational energy $\frac{1}{2}\omega L = 0.8$ matches Out[8]. The reported $|dE/E| = 8e-1$ is not an error — it honestly reports that the torque pumped energy in.

9.7 Example 7 — fixing a typo in an old cell with %edit

```

In[1]:= new sphere { mass = 20, radius = 0.5 }
Out[1]= obj0
In[2]:= set obj0.velocity = [1, 0, 0]
In[3]:= get obj0.momentum
Out[3]= [20, 0, 0]
In[4]:= %edit 1 new sphere { mass = 2, radius = 0.5 }
In[4]:= new sphere { mass = 2, radius = 0.5 }
Out[4]= obj1
In[5]:= %history
In[1]:= new sphere { mass = 2, radius = 0.5 }

```

```

Out[1]= obj0
In[2]:= set obj0.velocity = [1, 0, 0]
In[3]:= get obj0.momentum
Out[3]= [20, 0, 0]
In[4]:= new sphere { mass = 2, radius = 0.5 }
Out[4]= obj1

```

What to notice. `%edit 1 ...` rewrites cell 1's stored input **and re-executes it as the next cell** — like editing a Jupyter cell and shift-entering again. Because state is cumulative, the re-executed `NEW` made a *second* object; the fat `obj0` is still there. For a clean rebuild use `%reset + %rerun` or `%load`; when a field tweak suffices, `set obj0.mass = 2`. `%history` shows the *edited* input but the *original* outputs — an audit trail of what actually ran.

9.8 Example 8 — checking the simulator against itself

```

In[1]:= new point { mass = 2, position = [1, 0, 0], velocity = [0, 3, 0] }
Out[1]= obj0
In[2]:= cross(obj0.position, obj0.momentum)
Out[2]= [0, 0, 6]
In[3]:= angmom
Out[3]= [0, 0, 6]
In[4]:= dot(obj0.position, obj0.velocity)
Out[4]= 0
In[5]:= norm(cross(obj0.position, obj0.momentum)) / (norm(obj0.position) * norm(obj0.
momentum))
Out[5]= 1

```

What to notice. Cell 2 computes $L = r \times p$ by hand and cell 3 confirms `ANGMOM` agrees. Cell 4 proves $r \perp v$; cell 5 computes $|r \times p|/(|r||p|) = \sin \theta = 1$ — the angle between r and p is 90° — inside one nested expression.

9.9 Example 9 — hand-built tensors and a quaternion orientation

```

In[1]:= new cuboid { mass = 1, inertia_tensor = [[2, 0, 0], [0, 3, 0], [0, 0, 4]],
orientation = [0.7071067811865476, 0, 0, 0.7071067811865476] }
Out[1]= obj0
In[2]:= get obj0.inverse_inertia_tensor
Out[2]= [[0.5, 0, 0], [0, 0.3333333333333333, 0], [0, 0, 0.25]]
In[3]:= set obj0.angular_velocity = [0, 2, 0]
In[4]:= get obj0.angular_momentum
Out[4]= [0.0000000000000004440892098500628, 4.0000000000000002, 0]
In[5]:= get obj0.orientation
Out[5]= quat[w=0.7071067811865476, x=0, y=0, z=0.7071067811865476]

```

What to notice. Because `inertia_tensor` appears in the initializer, the analytic inertia is **not** recomputed — your `diag(2,3,4)` is kept, with the inverse maintained automatically. The quaternion $[w, x, y, z] = [\sqrt{1/2}, 0, 0, \sqrt{1/2}]$ is a 90° rotation about z , so setting the *world* angular velocity $[0, 2, 0]$ exercises the full $L = (RIR^T)\omega$ transformation. By hand: rotating `diag(2,3,4)` by 90° about z swaps the x and y moments, so $RIR^T = \text{diag}(3,2,4)$, the world- y moment is 2, and $L = [0, 4, 0]$ — exactly `Out[4]` (up to 4×10^{-16} of floating-point dust).

9.10 Example 10 — reproducibility: %save, %reset, %load

```
In[1]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= step 0.5
Out[3]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[4]:= %save session.posim
saved 3 cell(s) to session.posim
In[5]:= %reset
system reset
In[6]:= list
Out[6]= (no objects)
In[7]:= %load session.posim
In[7]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[7]= obj0
In[8]:= set system.gravity = [0, -9.81, 0]
In[9]:= step 0.5
Out[9]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[10]:= get obj0.velocity
Out[10]= [1, -4.905000000000001, 0]
```

What to notice. %save writes only the successful, non-magic inputs — a clean, replayable script. After %reset, %load replays it and the state is bit-identical ($v_y = -9.81 \cdot 0.5 = -4.905$). The same file runs headlessly via `posim -script`. The machine mode speaks the same language over JSON:

```
$ printf '%s\n' '{"op":"exec","code":"new point { mass = 1, velocity = [0, 1, 0] }"}' \
                '{"op":"get","path":"obj0.momentum"}' \
                '{"op":"set","path":"obj0.mass","value":5}' \
                '{"op":"get","path":"obj0.momentum"}' | posim --machine
{"display":"obj0","ok":true,"result":"obj0"}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}
{"display":"","ok":true,"result":null}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}
```

Changing the mass did **not** change the momentum — momentum is the canonical stored state; the *velocity* is what changed. That final subtlety is the union design showing through the wire protocol.

9.11 Example 11 — watching a Kepler orbit live, then running it backward

The same two-body setup as Example 1 — but this time we *watch* it. SCENE CREATE opens the scene window in the browser; SCENE START sets it in motion; SCENE REVERSE runs the movie backward.

```
In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= scene create 7878
```

```

Out[5]= scene window created: http://127.0.0.1:7878/
(opened in your browser; if no window appeared, open that address yourself)
showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[6]:= scene set_time_step 0.005
Out[6]= scene time step dt = 0.005
In[7]:= scene start
Out[7]= scene playback: running
In[8]:= scene pause
Out[8]= scene playback: paused
In[9]:= scene status
Out[9]= scene: http://127.0.0.1:7878/ (1 window(s) connected)
mode = paused, t = 0.4550000000000003, dt = 0.005, steps = 91, history = 91 frame(s)
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[10]:= scene reverse
Out[10]= scene playback: reversing
In[11]:= scene status
Out[11]= scene: http://127.0.0.1:7878/ (1 window(s) connected)
mode = reversing, t = 0.15500000000000005, dt = 0.005, steps = 91, history = 31 frame(s)
)
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[12]:= scene events
Out[12]= window connected (1 total)
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)

```

What to notice. Between In[7] and In[8] a few real seconds passed while the orbit ran in the window — playback happens on a background thread at ~ 30 frames per second, so **the notebook prompt never blocks**: cell 9’s STATUS shows the *playback copy* has advanced 91 steps to $t \approx 0.455$ while your notebook state still sits untouched at $t = 0$ (that is the “playback copy” design: the window animates its own synchronized copy of your system, so `GET system.time` here would still print 0). After REVERSE (cell 10), the status in cell 11 shows time flowing *backward* ($t \approx 0.155$) and the history buffer draining ($91 \rightarrow 31$ frames): each reversed frame is an exact recorded snapshot, so the rewind retraces the orbit bit-for-bit. Cell 12 drains the asynchronous event queue — the window announced itself when the browser connected. In JupyterLab you would not even need cell 12: events surface on their own as `[scene] ... lines (§7)`.

9.12 Example 12 — driving the camera from the notebook

Every gesture the mouse can make in the window has a command twin, so a *script* can compose the exact view you want — useful for repeatable demonstrations and screenshots.

```

In[1]:= new sphere { mass = 3, radius = 0.6, position = [2, 0, 0] }
Out[1]= obj0
In[2]:= new cuboid { mass = 1, half_extents = [0.4, 0.3, 0.2], position = [-2, 0, 1] }
Out[2]= obj1
In[3]:= scene create 7878
Out[3]= scene window created: http://127.0.0.1:7878/
(opened in your browser; if no window appeared, open that address yourself)
showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[4]:= scene translate 2 0

```

```

Out[4]= camera target = [2, 0, 0]
In[5]:= scene rotate 30 -10
Out[5]= camera yaw = -30°, pitch = 45°
In[6]:= scene zoom in
Out[6]= camera distance = 9.6
In[7]:= scene zoom 2
Out[7]= camera distance = 4.8
In[8]:= scene zoom out
Out[8]= camera distance = 5.999999999999999
In[9]:= scene hide 0
Out[9]= 1 object(s) hidden
In[10]:= scene show all
Out[10]= 0 object(s) hidden
In[11]:= scene redraw
Out[11]= scene redraw queued for every window
In[12]:= scene status
Out[12]= scene: http://127.0.0.1:7878/ (0 window(s) connected)
mode = stopped, t = 0, dt = 0.01, steps = 0, history = 0 frame(s)
entities = 2 (hidden: none)
camera: yaw = -30°, pitch = 45°, dist = 5.999999999999999, target = [2, 0, 0]
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)

```

What to notice. The camera is an **orbit camera**: it circles a *look-at point* at a *distance*, aimed by *yaw* (compass direction) and *pitch* (elevation). `TRANSLATE 2 0` moves the look-at point onto the sphere (this is what the arrow keys do in the window); `ROTATE 30 -10` adds 30° of yaw to the default -60° and -10° of pitch to the default 55° (this is what left-dragging does); the three zooms multiply the distance by 1/1.25, then 1/2, then 1.25 — watch the arithmetic: $12 \rightarrow 9.6 \rightarrow 4.8 \rightarrow 6$ (this is the mouse wheel). `HIDE 0` blanks the sphere out of every connected window without deleting anything — `SHOW ALL` brings it back. Because this transcript ran headless, cell 12 reports 0 window(s) connected — the commands work regardless, and any window that connects later receives the composed view. Note also the two-argument form `scene translate 2 0`: the optional `dz` defaulted to 0, and -10 in cell 5 was one negative argument, not a subtraction (scene arguments are *terms* — the second subtlety of §3).

9.13 Example 13 — the box of shapes: every body type in a rigid, infinitely massive box

The finale scene: `BOX 4` builds the rigid room (§5.7) and one of every shape goes inside ($R = 1$, $M = 1$) — a **torus** (mass M , inner radius 1, outer radius 2), a **point** (M , the only mover: $v = (100, 200, 100)$), a **sphere** ($2M$, $r = \frac{1}{2}$), an ideal zero-thickness **disk** ($2M/3$, $r = 1$), a **cube** ($5M/3$, side 1) and a **cylinder** ($2M$, $r = \frac{1}{2}$, height $3/2$). An axis-aligned torus of outer radius 2 would *exactly inscribe* the 4-box (its outer equator touching four walls), so the torus is tilted with its axis along $(1, 1, 1)/\sqrt{3}$: its extent per axis is $1.5\sqrt{2/3} + 0.5 \approx 1.7247 < 2$ — clearance 0.2753. The other positions are random, drawn by a documented LCG ($x \leftarrow 1664525x + 1013904223 \bmod 2^{32}$, seed 20260724, sequential rejection at ≥ 0.05 separation) — see `scripts/collisions/11_box_of_shapes.posim`, the script this transcript replays.

```

In[1]:= set system.g_constant = 0
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,

```

```

    obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
    the inside faces
In[3]:= new torus { mass = 1, inner_radius = 1, outer_radius = 2, orientation =
    [0.888073833977115, -0.325057583671868, 0.325057583671868, 0] }
Out[3]= obj6
In[4]:= new point { mass = 1, position = [1.406590, -0.995859, 0.569601], velocity =
    [100, 200, 100] }
Out[4]= obj7
In[5]:= new sphere { mass = 2, radius = 1/2, position = [1.424704, 1.367496, -0.493612]
    }
Out[5]= obj8
In[6]:= new disk { mass = 2/3, radius = 1, position = [-0.677386, -1.041493,
    -1.091679], orientation = [0.900447102352677, 0.307567078752479,
    -0.307567078752479, 0] }
Out[6]= obj9
In[7]:= new cuboid { mass = 5/3, half_extents = [0.5, 0.5, 0.5], position = [1.074397,
    0.816223, 1.102099] }
Out[7]= obj10
In[8]:= new cylinder { mass = 2, radius = 1/2, height = 3/2, position = [-1.027024,
    -1.403890, -0.485619], orientation = [0.968912421710645, 0, 0.247403959254523, 0] }
Out[8]= obj11
In[9]:= list
Out[9]= obj0: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[3, 0, 0] [wall: static,
    inverse_mass=0]
obj1: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[-3, 0, 0] [wall: static, inverse_mass
    =0]
obj2: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, 3, 0] [wall: static, inverse_mass
    =0]
obj3: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, -3, 0] [wall: static, inverse_mass
    =0]
obj4: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, 3] [wall: static, inverse_mass
    =0]
obj5: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, -3] [wall: static, inverse_mass
    =0]
obj6: torus ring=1.5 tube=0.5, mass=1, charge=0, pos=[0, 0, 0]
obj7: point, mass=1, charge=0, pos=[1.40659, -0.995859, 0.569601]
obj8: sphere r=0.5, mass=2, charge=0, pos=[1.424704, 1.367496, -0.493612]
obj9: disk r=1, mass=0.6666666666666666, charge=0, pos=[-0.677386, -1.041493,
    -1.091679]
obj10: cuboid he=[0.5, 0.5, 0.5], mass=1.6666666666666667, charge=0, pos=[1.074397,
    0.816223, 1.102099]
obj11: cylinder r=0.5 h=1.5, mass=2, charge=0, pos=[-1.027024, -1.40389, -0.485619]
In[10]:= collide
Out[10]= collisions ON (51 collidable pair(s); 0 impulse(s) so far)
In[11]:= get obj0.inverse_mass
Out[11]= 0
In[12]:= get obj6.inertia_tensor
Out[12]= [[1.28125, 0, 0], [0, 1.28125, 0], [0, 0, 2.4375]]
In[13]:= energy
Out[13]= 30000
In[14]:= momentum
Out[14]= [100, 200, 100]
In[15]:= run 0.1 steps 100
Out[15]= t = 0.1 (2121 solver steps, 100 snapshots, |dE/E| = 2.040e-10, 119 collision(s)

```

```

) -- CONTACTS lists them)
In[16]:= energy
Out[16]= 29999.99999388012
In[17]:= momentum
Out[17]= [146.4891109312449, 102.14781200712048, 46.01601279520784]
In[18]:= get system.collisions
Out[18]= 132
In[19]:= get system.box
Out[19]= 4

```

What to notice. Cell 3 sizes the torus by the order-independent `inner_radius + outer_radius` pair; cell 8 uses `height = 3/2` — the *full* height, so `half_height` reads 0.75. COLLIDE counts **51 collidable pairs**: 12 objects make 66 pairs, minus the 15 wall–wall pairs (two static bodies can never collide). `Out[12]` is the torus’s analytic inertia `diag(1.28125, 1.28125, 2.4375)` — exactly $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$, $I_z = m(c^2 + \frac{3}{4}a^2)$ for $c = 1.5$, $a = 0.5$. The energy starts at $E_0 = \frac{1}{2} \cdot 1 \cdot |v|^2 = 30000$ **exactly** and, after **119 collisions** in 0.1 s, is conserved to $|dE/E| = 1.031 \times 10^{-9}$ — but momentum went from (100, 200, 100) to (146.49, 102.15, 46.02): **not conserved**, because the infinitely massive walls absorb momentum without moving — the physical signature of infinite mass. Along the way the point particle can *thread the torus hole* and passes through the ideal zero-thickness disk (a measure-zero contact) — the ball-vs-anything collision tier is exact SDF geometry, not an approximation — while every finite-size body bounces off both. The walls end the run bit-identically at rest.

10 Error message tour

you type	you get
<code>get obj7.mass</code> (no <code>obj7</code>)	no object <code>obj7</code>
<code>get obj0.bogus</code>	unknown object field ‘bogus’ -- see HELP for the field list
<code>[1,0,0] * [0,1,0]</code>	cannot multiply <code>vec3</code> by <code>vec3</code> (for <code>vec3*vec3</code> use <code>dot()/cross()</code>)
<code>set = 3</code>	parse error at column 5: expected a path root (<code>objN</code> or <code>system</code>), found =
<code>run 1</code> under SPRK with $B \neq 0$	SPRK method requires a separable Hamiltonian: magnetic field <code>B</code> must be zero ... use METHOD ADAMS or BDF
<code>bogusname</code>	parse error ...: unknown name <code>bogusname</code> (expected a number, <code>[x,y,z]</code> , <code>objN.field</code> , <code>system.field</code> , or a function call)

Every error names the column or the exact field/feature at fault, and never aborts the session.